

Building standalone EpisodeSleuth executables

EpisodeSleuth can be packaged into a **self-contained executable** that runs the Fluent desktop GUI **without Python installed** on the target machine. This uses [PyInstaller](#) and a single cross-platform build spec, `episodesleuth.spec`, driven by a small wrapper script per OS.

There are three ways to ship the app, from simplest to most polished:

Method	Best for	Script
Standalone executable (this doc)	Handing someone a runnable app to double-click; Linux distribution	<code>build_exe.ps1</code> / <code>build_exe.sh</code>
MSIX installer	Windows Store / signed enterprise deployment	<code>build_msix.ps1</code>
Source install	Developers, or users who already have Python	<code>install.ps1</code> / <code>pip install -e .</code>

Quick start

Windows

```
powershell -ExecutionPolicy Bypass -File .\build_exe.ps1
```

Output: `dist\EpisodeSleuth\EpisodeSleuth.exe`. Zip the whole `dist\EpisodeSleuth` folder to share it - the recipient unzips and double-clicks `EpisodeSleuth.exe`, no Python required.

Linux / macOS

```
./build_exe.sh
```

Output: `dist/EpisodeSleuth/EpisodeSleuth`. Distribute the whole `dist/EpisodeSleuth` folder.

Both scripts build inside a throwaway `.buildvenv` so your system Python stays clean, then run `PyInstaller` against `episodesleuth.spec`.

Build options

Both wrappers accept the same switches (via flags on Windows, environment variables on Linux/macOS):

Option	Windows	Linux/macOS	Effect
Bundle the Vosk model	<code>-BundleModel</code>	<code>BUNDLE_MODEL=1 ./build_exe.sh</code>	Packs <code>models\</code> into the bundle for a fully offline app (much larger). Off by default - the model is downloaded on first run / by <code>install.ps1</code> .
Single-file exe	<code>-OneFile</code>	<code>ONEFILE=1 ./build_exe.sh</code>	Produces one <code>EpisodeSleuth[.exe]</code> file instead of a one-folder bundle. Simpler to share but slower to start (it unpacks to a temp dir each launch).

Example - fully offline single-file Windows build:

```
powershell -ExecutionPolicy Bypass -File .\build_exe.ps1 -BundleModel -OneFile
```

What ends up in the bundle

`episodesleuth.spec` deliberately keeps the bundle lean:

- **Included:** the `audio_fingerprint` package (engine + CLI + GUI), `PySide6` + `PySide6-Fluent-Widgets`, `Vosk`, `metaphone`, `Levenshtein`, `NumPy`, and `config.json` + the app icon (`packaging/app.ico`).
- **Excluded:** heavy scientific / ML / cloud packages that may exist in your build environment but are never imported by `EpisodeSleuth` (`torch`, `tensorflow`, `pyarrow`, `opencv`, `scipy`, `pandas`, `matplotlib`, `boto3`, `jupyter`, other Qt bindings, `tkinter`, ...). Without these excludes the bundle balloons to ~1.7 GB; with them a normal one-folder build is roughly **~400 MB** (dominated by `PySide6/Qt` and `Vosk`).

The entry point is `gui/__main__.py` (the same `python -m audio_fingerprint.gui` target), and the spec adds both the project root and its parent to `pathex` so the mixed bare imports (`gui_config` , `engine` , `fingerprint_core`) and package-qualified imports (`audio_fingerprint.gui.*`) all resolve.

Prerequisites

- **Python 3.9+** on `PATH` (only on the build machine - not on the machines that run the finished executable).
- **FFmpeg** is still required at **run time** for decoding audio. It is an external tool and is intentionally not bundled; install it with `winget install Gyan.FFmpeg` (Windows) or your package manager (Linux/macOS), or run `install.ps1` first, which sets it up.

The wrapper scripts install PyInstaller and the project's `requirements.txt` into `.buildvenv` automatically.

Running / distributing

- **One-folder build (default):** distribute the entire `dist/EpisodeSleuth` folder (zip it). The executable and its `_internal` support files must stay together.
- **One-file build (`-OneFile`):** distribute the single `EpisodeSleuth[.exe]` .

On first run, if the Vosk speech model was not bundled, use the app's **Settings** page to download it (or run `install.ps1`).

Relationship to the MSIX build

`build_msix.ps1` performs the same PyInstaller step and then wraps the result in a signed `.msix` using `packaging\AppxManifest.xml` . Use `build_exe.ps1` when you just want a runnable folder/exe, and `build_msix.ps1` when you need a proper Windows installer / Store package. Both share `episodesleuth.spec` conceptually, so the runtime behavior is identical.